TRD

Technical Requirements Document (TRD)

AWS S3-Based Data Processing Pipeline

•

TR-INGEST-001

• Related Functional Requirement(s): FR-INGEST-001

• Objective:

Implement S3-to-Spark ingestion for customer CSV files using AWS Glue Spark job.

• Target Cluster Configuration:

• AWS Glue ETL Job (Spark)

• Glue Version: 3.0 or higher (supports Spark 3.1+)

• Worker Type: G.1X or G.2X

• Number of Workers: Configurable based on data volume (minimum 2)

• Python Shell: Python 3

• Source Data Details:

• Location: `s3://adif-sdlc/sdlc_wizard/customerdata/`

• Format: CSV

• Delimiter: Comma (`,`)

• Header: Present (first row)

• Schema:

• `CustId` (String)

• `Name` (String)

• `EmailId` (String)

• `Region` (String)

• File Pattern: `.csv`

• Target Data Details:

• In-memory Spark DataFrame (no persistence at this stage)

• Job Flow / Pipeline Stages:

1. Initialize Glue Spark context

2. Read CSV files from S3 path using `spark.read.csv()`

3. Apply schema inference or explicit schema definition

4. Load into DataFrame named `customer_df`

• Data Transformations / Business Logic:

• Parse CSV with header=True

• Infer schema or apply explicit StructType with fields: CustId, Name, EmailId, Region

• No transformations applied at ingestion stage

• Error Handling and Logging:

• Log file count and row count to CloudWatch

• Catch and log S3 access errors (NoSuchBucket, AccessDenied)

• Fail job if no files found or schema mismatch detected

• Log DataFrame schema to CloudWatch for validation

• Assumptions:

• All CSV files in the path are valid customer data files

• Files use UTF-8 encoding

• No nested or multi-line CSV records

• IAM role attached to Glue job has `s3:GetObject` and `s3:ListBucket` permissions

•

TR-INGEST-002

• Related Functional Requirement(s): FR-INGEST-002

• Objective:

Implement S3-to-Spark ingestion for order CSV files using AWS Glue Spark job.

• Target Cluster Configuration:

• AWS Glue ETL Job (Spark)

• Glue Version: 3.0 or higher

• Worker Type: G.1X or G.2X

• Number of Workers: Configurable based on data volume (minimum 2)

• Python Shell: Python 3

• Source Data Details:

• Location: `s3://adif-sdlc/sdlc_wizard/orderdata/`

• Format: CSV

• Delimiter: Comma (`,`)

• Header: Present (first row)

• Schema:

• `OrderId` (String)

• `ItemName` (String)

• `PricePerUnit` (Decimal/Double)

• `Qty` (Integer)

• `Date` (String, parseable to Date)

• `CustId` (String)

• File Pattern: `.csv`

• Target Data Details:

• In-memory Spark DataFrame (no persistence at this stage)

• Job Flow / Pipeline Stages:

1. Initialize Glue Spark context

2. Read CSV files from S3 path using `spark.read.csv()`

3. Apply schema inference or explicit schema definition

4. Load into DataFrame named `order_df`

• Data Transformations / Business Logic:

• Parse CSV with header=True

• Infer schema or apply explicit StructType with fields: OrderId, ItemName, PricePerUnit, Qty, Date, CustId

• Cast PricePerUnit to DoubleType, Qty to IntegerType

• No transformations applied at ingestion stage

• Error Handling and Logging:

• Log file count and row count to CloudWatch

• Catch and log S3 access errors (NoSuchBucket, AccessDenied)

• Fail job if no files found or schema mismatch detected

• Log DataFrame schema to CloudWatch for validation

• Assumptions:

• All CSV files in the path are valid order data files

• Files use UTF-8 encoding

• Date field is in a standard format (e.g., YYYY-MM-DD or MM/DD/YYYY)

• IAM role attached to Glue job has `s3:GetObject` and `s3:ListBucket` permissions

•

TR-CLEAN-001

- Related Functional Requirement(s): FR-CLEAN-001

- Objective:

Remove rows containing literal string "Null" or Spark NULL values from customer DataFrame.

- Target Cluster Configuration:
- Same Glue Spark job as TR-INGEST-001
- In-memory DataFrame transformation

- Source Data Details:
- In-memory DataFrame `customer_df` from TR-INGEST-001
- Schema: CustId, Name, EmailId, Region

- Target Data Details:
- Cleaned in-memory DataFrame `customer_cleaned_df`

- Job Flow / Pipeline Stages:

1. Receive `customer_df` from ingestion stage
2. Apply null filtering logic
3. Output `customer_cleaned_df` for deduplication stage

- Data Transformations / Business Logic:
- Filter out rows where any column equals literal string "Null" (case-sensitive)
- Filter out rows where any column is Spark NULL
- Implementation:

```
customer_cleaned_df = customer_df.filter(
(col("CustId") != "Null") & (col("CustId").isNotNull()) &
(col("Name") != "Null") & (col("Name").isNotNull()) &
(col("EmailId") != "Null") & (col("EmailId").isNotNull()) &
(col("Region") != "Null") & (col("Region").isNotNull())
)
```

- Error Handling and Logging:
- Log row count before and after cleaning to CloudWatch
- Log number of rows removed

• No failure condition (empty result is valid)

• Assumptions:

• "Null" is case-sensitive (exact match)

• Entire row is dropped if any column contains "Null" or NULL

• No partial row cleaning or imputation

•

TR-CLEAN-002

• Related Functional Requirement(s): FR-CLEAN-002

• Objective:

Remove rows containing literal string "Null" or Spark NULL values from order DataFrame.

• Target Cluster Configuration:

• Same Glue Spark job as TR-INGEST-002

• In-memory DataFrame transformation

• Source Data Details:

• In-memory DataFrame `order_df` from TR-INGEST-002

• Schema: OrderId, ItemName, PricePerUnit, Qty, Date, CustId

• Target Data Details:

• Cleaned in-memory DataFrame `order_cleaned_df`

• Job Flow / Pipeline Stages:

1. Receive `order_df` from ingestion stage

2. Apply null filtering logic

3. Output `order_cleaned_df` for deduplication stage

• Data Transformations / Business Logic:

• Filter out rows where any column equals literal string "Null" (case-sensitive)

• Filter out rows where any column is Spark NULL

• Implementation:

```
order_cleaned_df = order_df.filter(
(col("OrderId") != "Null") & (col("OrderId").isNotNull()) &
(col("ItemName") != "Null") & (col("ItemName").isNotNull()) &
(col("PricePerUnit") != "Null") & (col("PricePerUnit").isNotNull()) &
```

```
(col("Qty") != "Null") & (col("Qty").isNotNull()) &
(col("Date") != "Null") & (col("Date").isNotNull()) &
(col("CustId") != "Null") & (col("CustId").isNotNull())
)
```

- Error Handling and Logging:
- Log row count before and after cleaning to CloudWatch
- Log number of rows removed
- No failure condition (empty result is valid)

- Assumptions:
- "Null" is case-sensitive (exact match)
- Entire row is dropped if any column contains "Null" or NULL
- No partial row cleaning or imputation

-

TR-CLEAN-003

- Related Functional Requirement(s): FR-CLEAN-003

- Objective:

Remove duplicate rows from customer DataFrame, retaining first occurrence.

- Target Cluster Configuration:
- Same Glue Spark job as TR-CLEAN-001
- In-memory DataFrame transformation

- Source Data Details:
- In-memory DataFrame `customer_cleaned_df` from TR-CLEAN-001
- Schema: CustId, Name, EmailId, Region

- Target Data Details:
- Deduplicated in-memory DataFrame `customer_final_df`

- Job Flow / Pipeline Stages:

1. Receive `customer_cleaned_df` from null removal stage
2. Apply deduplication logic
3. Output `customer_final_df` for catalog registration

• Data Transformations / Business Logic:

• Apply `dropDuplicates()` across all columns

• Retain first occurrence by default Spark behavior

• Implementation:

```
customer_final_df = customer_cleaned_df.dropDuplicates()
```

• Error Handling and Logging:

• Log row count before and after deduplication to CloudWatch

• Log number of duplicate rows removed

• No failure condition (empty result is valid)

• Assumptions:

• Duplicates are identified by comparing all column values

• First occurrence is retained (Spark default behavior)

• No custom deduplication key or logic required

•

TR-CLEAN-004

• Related Functional Requirement(s): FR-CLEAN-004

• Objective:

Remove duplicate rows from order DataFrame, retaining first occurrence.

• Target Cluster Configuration:

• Same Glue Spark job as TR-CLEAN-002

• In-memory DataFrame transformation

• Source Data Details:

• In-memory DataFrame `order_cleaned_df` from TR-CLEAN-002

• Schema: OrderId, ItemName, PricePerUnit, Qty, Date, CustId

• Target Data Details:

• Deduplicated in-memory DataFrame `order_final_df`

• Job Flow / Pipeline Stages:

1. Receive `order_cleaned_df` from null removal stage

2. Apply deduplication logic

3. Output `order_final_df` for catalog registration

• Data Transformations / Business Logic:

• Apply `dropDuplicates()` across all columns

• Retain first occurrence by default Spark behavior

• Implementation:

```
order_final_df = order_cleaned_df.dropDuplicates()
```

• Error Handling and Logging:

• Log row count before and after deduplication to CloudWatch

• Log number of duplicate rows removed

• No failure condition (empty result is valid)

• Assumptions:

• Duplicates are identified by comparing all column values

• First occurrence is retained (Spark default behavior)

• No custom deduplication key or logic required

•

TR-CATALOG-001

• Related Functional Requirement(s): FR-CATALOG-001

• Objective:

Write cleaned customer DataFrame to AWS Glue Data Catalog as a managed table.

• Target Cluster Configuration:

• Same Glue Spark job as TR-CLEAN-003

• Glue Catalog write operation

• Source Data Details:

• In-memory DataFrame `customer_final_df` from TR-CLEAN-003

• Schema: CustId, Name, EmailId, Region

• Target Data Details:

• Database: `gen_ai_poc_databrickscoe`

• Table Name: `sdlc_wizard_customer`

• Storage Format: Parquet (default for Glue Catalog)

• Write Mode: Overwrite (or append based on job parameter)

• Partition: None specified

• Job Flow / Pipeline Stages:

1. Receive `customer_final_df` from deduplication stage

2. Create Glue DynamicFrame from Spark DataFrame

3. Write to Glue Catalog using `glueContext.write_dynamic_frame.from_catalog()`

• Data Transformations / Business Logic:

• Convert Spark DataFrame to Glue DynamicFrame

• No schema transformation required

• Implementation:

```
customer_dynamic_frame = DynamicFrame.fromDF(customer_final_df, glueContext, "customer_dynamic_frame")
glueContext.write_dynamic_frame.from_catalog(
frame=customer_dynamic_frame,
database="gen_ai_poc_databrickscoe",
table_name="sdlc_wizard_customer",
transformation_ctx="write_customer_catalog"
)
```

• Error Handling and Logging:

• Log successful catalog write to CloudWatch

• Catch and log catalog write errors (AccessDenied, DatabaseNotFound)

• Fail job if catalog write fails

• Log final row count written to catalog

• Assumptions:

• Database `gen_ai_poc_databrickscoe` exists prior to job execution

• IAM role has `glue:CreateTable`, `glue:UpdateTable`, `glue:GetTable` permissions

• Table is created if it does not exist; overwritten if it exists (based on write mode)

• Default storage format is Parquet

•

TR-CATALOG-002

• Related Functional Requirement(s): FR-CATALOG-002

• Objective:

Write cleaned order DataFrame to AWS Glue Data Catalog as a managed table.

• Target Cluster Configuration:

• Same Glue Spark job as TR-CLEAN-004

• Glue Catalog write operation

• Source Data Details:

• In-memory DataFrame `order_final_df` from TR-CLEAN-004

• Schema: OrderId, ItemName, PricePerUnit, Qty, Date, CustId

• Target Data Details:

• Database: `gen_ai_poc_databrickscoe`

• Table Name: `sdlc_wizard_order`

• Storage Format: Parquet (default for Glue Catalog)

• Write Mode: Overwrite (or append based on job parameter)

• Partition: None specified

• Job Flow / Pipeline Stages:

1. Receive `order_final_df` from deduplication stage

2. Create Glue DynamicFrame from Spark DataFrame

3. Write to Glue Catalog using `glueContext.write_dynamic_frame.from_catalog()`

• Data Transformations / Business Logic:

• Convert Spark DataFrame to Glue DynamicFrame

• No schema transformation required

• Implementation:

```
order_dynamic_frame = DynamicFrame.fromDF(order_final_df, glueContext, "order_dynamic_frame")
glueContext.write_dynamic_frame.from_catalog(
frame=order_dynamic_frame,
database="gen_ai_poc_databrickscoe",
table_name="sdlc_wizard_order",
transformation_ctx="write_order_catalog"
)
```

- Error Handling and Logging:
- Log successful catalog write to CloudWatch
- Catch and log catalog write errors (AccessDenied, DatabaseNotFound)
- Fail job if catalog write fails
- Log final row count written to catalog

- Assumptions:
- Database `gen_ai_poc_databrickscoe` exists prior to job execution
- IAM role has `glue:CreateTable`, `glue:UpdateTable`, `glue:GetTable` permissions
- Table is created if it does not exist; overwritten if it exists (based on write mode)
- Default storage format is Parquet
-

TR-SCD2-001

- Related Functional Requirement(s): FR-SCD2-001

- Objective:

Implement SCD Type 2 logic for `ordersummary` table using Apache Hudi on S3.

- Target Cluster Configuration:
- AWS Glue ETL Job (Spark) with Hudi support
- Glue Version: 3.0 or higher
- Worker Type: G.1X or G.2X
- Number of Workers: Configurable based on data volume (minimum 2)
- Additional JARs: Hudi Spark bundle (included in Glue 3.0+)

- Source Data Details:
- Customer Table: `gen_ai_poc_databrickscoe.sdlc_wizard_customer`
- Order Table: `gen_ai_poc_databrickscoe.sdlc_wizard_order`
- Existing Hudi Table (if exists): `s3://adif-sdlc/curated/sdlc_wizard/ordersummary/`

- Target Data Details:
- Location: `s3://adif-sdlc/curated/sdlc_wizard/ordersummary/`
- Format: Apache Hudi (Copy-on-Write or Merge-on-Read)
- Table Type: Hudi Merge-on-Read (MOR) recommended for SCD2
- Schema:
- All fields from customer-order join
- `IsActive` (Boolean)

• `StartDate` (Timestamp)

• `EndDate` (Timestamp, nullable)

• `OpTs` (Timestamp)

• Primary Key: Composite key derived from CustId + OrderId (or business key)

• Precombine Field: `OpTs`

• Job Flow / Pipeline Stages:

1. Read customer and order tables from Glue Catalog

2. Join customer and order DataFrames on `CustId`

3. Read existing Hudi `ordersummary` table (if exists)

4. Identify new and changed records by comparing joined data with existing table

5. For changed records:

• Update existing active row: set `IsActive = False`, `EndDate = current_timestamp()`

• Insert new row: set `IsActive = True`, `StartDate = current_timestamp()`, `EndDate = NULL`

6. For new records:

• Insert with `IsActive = True`, `StartDate = current_timestamp()`, `EndDate = NULL`

7. Set `OpTs = current_timestamp()` for all operations

8. Write to Hudi table using upsert operation

• Data Transformations / Business Logic:

• Join Logic:

```
joined_df = customer_final_df.join(order_final_df, "CustId", "inner")
```

• SCD2 Logic:

• Compare joined_df with existing Hudi table on primary key

• Detect changes by comparing all non-SCD2 columns

• Generate two records for changed rows: one to close old record, one to insert new record

• Generate one record for new rows

• Column Additions:

• Add `IsActive` (Boolean): True for new/updated records

• Add `StartDate` (Timestamp): `current_timestamp()` for new/updated records

• Add `EndDate` (Timestamp): NULL for active records, `current_timestamp()` for closed records

• Add `OpTs` (Timestamp): `current_timestamp()` for all records

• Hudi Write Configuration:

```
```

```
hudi_options = {
'hoodie.table.name': 'ordersummary',
'hoodie.datasource.write.recordkey.field': 'CustId,OrderId',
'hoodie.datasource.write.precombine.field': 'OpTs',
'hoodie.datasource.write.operation': 'upsert',
'hoodie.datasource.write.table.type': 'MERGE_ON_READ',
'hoodie.upsert.shuffle.parallelism': 2,
'hoodie.insert.shuffle.parallelism': 2
}
```

- Error Handling and Logging:
- Log join row count to CloudWatch
- Log number of new, changed, and unchanged records
- Catch and log Hudi write errors
- Fail job if Hudi write fails
- Log Hudi commit metadata to CloudWatch

- Assumptions:
- Primary key for SCD2 is composite: `CustId + OrderId`
- Change detection compares all non-SCD2 columns (CustId, Name, EmailId, Region, OrderId, ItemName, PricePerUnit, Qty, Date)
- `OpTs` is used as precombine field for Hudi upsert conflict resolution
- `EndDate` is NULL for active records
- Hudi table is initialized on first run if it does not exist
- IAM role has `s3:PutObject`, `s3:GetObject`, `s3:DeleteObject` permissions on target S3 path
-

TR-SCD2-002

- Related Functional Requirement(s): FR-SCD2-002

- Objective:

Register Hudi `ordersummary` table in AWS Glue Data Catalog.

- Target Cluster Configuration:
- Same Glue Spark job as TR-SCD2-001
- Glue Catalog registration operation

• Source Data Details:

• Hudi table at `s3://adif-sdlc/curated/sdlc_wizard/ordersummary/`

• Target Data Details:

• Database: `gen_ai_poc_databrickscoe`

• Table Name: `ordersummary`

• Storage Format: Hudi

• Table Type: External table pointing to Hudi S3 location

• Job Flow / Pipeline Stages:

1. Complete Hudi write operation from TR-SCD2-001

2. Register Hudi table in Glue Catalog using Glue API or Spark SQL

3. Verify table is queryable via Athena

• Data Transformations / Business Logic:

• Use Glue Catalog API or Spark SQL to create external table:

```
spark.sql(f"""
CREATE EXTERNAL TABLE IF NOT EXISTS gen_ai_poc_databrickscoe.ordersummary
USING hudi
LOCATION 's3://adif-sdlc/curated/sdlc_wizard/ordersummary/'
""")
```

• Alternatively, use `glueContext.create_dynamic_frame.from_options()` with Hudi format

• Error Handling and Logging:

• Log successful catalog registration to CloudWatch

• Catch and log catalog registration errors

• Fail job if catalog registration fails

• Verify table metadata in Glue Catalog

• Assumptions:

• Glue Catalog supports Hudi table format

• Athena version supports querying Hudi tables (Athena Engine v2 or v3)

• IAM role has `glue:CreateTable`, `glue:UpdateTable` permissions

• Table is created if it does not exist; updated if it exists

•

TR-INCR-001

- Related Functional Requirement(s): FR-INCR-001

- Objective:

Implement incremental SCD2 updates triggered by customer data changes.

- Target Cluster Configuration:
- AWS Glue ETL Job (Spark) with Hudi support
- Glue Version: 3.0 or higher
- Worker Type: G.1X or G.2X
- Number of Workers: Configurable based on data volume (minimum 2)
- Trigger: Scheduled (e.g., daily) or event-driven (S3 event notification)

- Source Data Details:
- Customer Table: `gen_ai_poc_databrickscoe.sdlc_wizard_customer`
- Order Table: `gen_ai_poc_databrickscoe.sdlc_wizard_order`
- Existing Hudi Table: `s3://adif-sdlc/curated/sdlc_wizard/ordersummary/`

- Target Data Details:
- Location: `s3://adif-sdlc/curated/sdlc_wizard/ordersummary/`
- Format: Apache Hudi
- Updated Schema: Same as TR-SCD2-001

- Job Flow / Pipeline Stages:
1. Read customer table from Glue Catalog
2. Identify changed or new customer records since last processing:
- Use Hudi incremental query on customer table (if customer table is also Hudi)
- Or compare customer table with previous snapshot using hash or timestamp
3. Join changed customer records with order table on `CustId`
4. Read existing Hudi `ordersummary` table
5. Apply SCD2 logic (same as TR-SCD2-001) only for affected records
6. Write updates to Hudi table using upsert operation

- Data Transformations / Business Logic:
- Change Detection:
- Calculate hash of customer record fields (Name, EmailId, Region)
- Compare with previous hash stored in metadata or separate tracking table
- Identify records where hash differs or CustId is new

• Incremental Join:

```
changed_customers_df = customer_final_df.filter()
incremental_joined_df = changed_customers_df.join(order_final_df, "CustId", "inner")
```

• SCD2 Logic: Same as TR-SCD2-001, applied only to `incremental_joined_df`

• Hudi Incremental Write: Use same Hudi options as TR-SCD2-001

• Error Handling and Logging:

• Log number of changed customer records detected

• Log number of affected order records

• Log number of SCD2 updates (closed + inserted rows)

• Catch and log Hudi write errors

• Fail job if incremental update fails

• Assumptions:

• Change detection uses hash comparison or timestamp-based logic

• Customer table includes a last-modified timestamp or version field for change detection

• Incremental processing runs on a scheduled basis (e.g., daily, hourly)

• Only customer changes trigger updates; order changes are handled separately (not specified in FRD)

• IAM role has necessary S3 and Glue permissions

•

TR-AGG-001

• Related Functional Requirement(s): FR-AGG-001

• Objective:

Generate aggregated customer spending table from `ordersummary` Hudi table.

• Target Cluster Configuration:

• AWS Glue ETL Job (Spark)

• Glue Version: 3.0 or higher

• Worker Type: G.1X or G.2X

• Number of Workers: Configurable based on data volume (minimum 2)

• Source Data Details:

• Hudi Table: `gen_ai_poc_databrickscoe.ordersummary`

• Filter: `IsActive = True`

• Target Data Details:

• Location: `s3://adif-sdlc/analytics/customeraggregatespend/`

• Format: Parquet (default for Glue Catalog)

• Schema:

• `Name` (String)

• `TotalAmount` (Double)

• `Date` (Date or Timestamp)

• Job Flow / Pipeline Stages:

1. Read `ordersummary` table from Glue Catalog
2. Filter for active records (`IsActive = True`)
3. Calculate `TotalAmount` per customer: `SUM(PricePerUnit Qty)` grouped by `Name`
4. Include `Date` field (latest order date per customer or aggregation timestamp)
5. Write result to S3 in Parquet format

• Data Transformations / Business Logic:

• Filter Active Records:

```
active_orders_df = ordersummary_df.filter(col("IsActive") == True)
```

• Aggregation:

```
aggregate_df = active_orders_df.groupBy("Name").agg(
sum(col("PricePerUnit") col("Qty")).alias("TotalAmount"),
max(col("Date")).alias("Date")
)
```

• Date Field: Use `max(Date)` to capture latest order date per customer, or use `current_date()` for aggregation date

• Error Handling and Logging:

• Log number of active records processed

• Log number of unique customers in aggregation

• Log total aggregated amount (sum of all TotalAmount)

• Catch and log S3 write errors

• Fail job if write fails

• Assumptions:

• `TotalAmount` is calculated as `SUM(PricePerUnit Qty)` per customer

• `Date` represents the latest order date for each customer

• Only active records from `ordersummary` are included

• Output format is Parquet for efficient querying

• IAM role has `s3:PutObject` permissions on target S3 path

•

TR-AGG-002

• Related Functional Requirement(s): FR-AGG-002

• Objective:

Register `customeraggregatespend` table in AWS Glue Data Catalog.

• Target Cluster Configuration:

• Same Glue Spark job as TR-AGG-001

• Glue Catalog write operation

• Source Data Details:

• In-memory DataFrame `aggregate_df` from TR-AGG-001

• Schema: Name, TotalAmount, Date

• Target Data Details:

• Database: `gen_ai_poc_databrickscoe`

• Table Name: `customeraggregatespend`

• Storage Format: Parquet

• Write Mode: Overwrite (or append based on job parameter)

• Partition: None specified

• Job Flow / Pipeline Stages:

1. Receive `aggregate_df` from aggregation stage

2. Create Glue DynamicFrame from Spark DataFrame

3. Write to Glue Catalog using `glueContext.write_dynamic_frame.from_catalog()`

• Data Transformations / Business Logic:

• Convert Spark DataFrame to Glue DynamicFrame

• No schema transformation required

• Implementation:

```
aggregate_dynamic_frame = DynamicFrame.fromDF(aggregate_df, glueContext, "aggregate_dynamic_frame")
glueContext.write_dynamic_frame.from_catalog(
frame=aggregate_dynamic_frame,
database="gen_ai_poc_databrickscoe",
table_name="customeraggregatespend",
transformation_ctx="write_aggregate_catalog"
)
```

• Error Handling and Logging:

• Log successful catalog write to CloudWatch

• Catch and log catalog write errors

• Fail job if catalog write fails

• Log final row count written to catalog

• Assumptions:

• Database `gen_ai_poc_databrickscoe` exists prior to job execution

• IAM role has `glue:CreateTable`, `glue:UpdateTable` permissions

• Table is created if it does not exist; overwritten if it exists (based on write mode)

• Default storage format is Parquet

•

TR-ARCH-001

• Related Functional Requirement(s): FR-ARCH-001

• Objective:

Configure AWS Glue ETL jobs for all data processing operations.

• Target Cluster Configuration:

• Service: AWS Glue

• Job Type: Spark ETL (Python Shell)

• Glue Version: 3.0 or higher

• Worker Type: G.1X (4 vCPU, 16 GB memory) or G.2X (8 vCPU, 32 GB memory)

• Number of Workers: Configurable (minimum 2, scale based on data volume)

• Max Concurrent Runs: 1 (or configurable based on requirements)

• Timeout: 2880 minutes (48 hours, configurable)

• Max Retries: 0 (or configurable)

• Script Location: S3 bucket for Glue scripts

• Temp Directory: S3 bucket for temporary files

• Additional Libraries: Hudi Spark bundle (included in Glue 3.0+)

• Source Data Details:

• N/A (architecture-level requirement)

• Target Data Details:

• N/A (architecture-level requirement)

• Job Flow / Pipeline Stages:

1. Create Glue job definitions for each ETL stage:

• Customer ingestion and cleaning (TR-INGEST-001, TR-CLEAN-001, TR-CLEAN-003, TR-CATALOG-001)

• Order ingestion and cleaning (TR-INGEST-002, TR-CLEAN-002, TR-CLEAN-004, TR-CATALOG-002)

• SCD2 processing (TR-SCD2-001, TR-SCD2-002)

• Incremental updates (TR-INCR-001)

• Aggregation (TR-AGG-001, TR-AGG-002)

2. Configure job parameters (S3 paths, database names, table names)

3. Set up IAM role with necessary permissions

4. Enable CloudWatch logging for all jobs

• Data Transformations / Business Logic:

• N/A (architecture-level requirement)